



From Blueprint to Production: Building Real AI Agents

What it took to run an LLM agent in production for multiple enterprise tenants, and what we would do again.

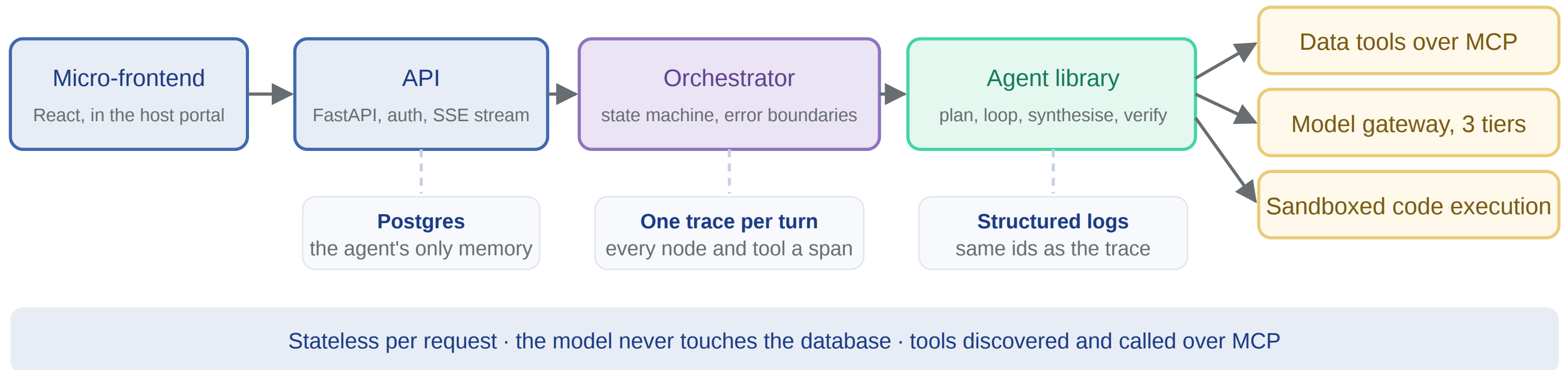
Blaise Paradeza, Senior AI Software Engineer

Jiggy Kakkad, Principal AI Engineer

Sydney AI Engineering Meetup, 23 September 2026

What we run

An enterprise conversational analytics agent. A business user asks a question in natural language; the agent plans, calls governed data tools, runs sandboxed analysis and streams back a verified, charted answer. Embedded in customer portals, one codebase, isolated tenant deployments.



- **Stack.** React micro-frontend, FastAPI, Postgres, Kubernetes. A LangGraph state machine over a Pydantic-typed agent library, structured output via pydantic-ai. Model gateway, Langfuse traces, Datadog logs and monitors.
- **Team.** A small squad, fortnightly releases. The product is an agent, and most of its code is written by an agent. Both need a harness, which is the thread of this talk.

OUR OPERATING ASSUMPTION

The agent **will** fail.
The harness **must** catch it.

You cannot prompt-engineer your way out of LLM unreliability. You cannot review your way out of LLM-generated code volume. The defensive layers are the engineering work. Three acts tonight:

ACT 1

The agent

What runs in production, and why it is shaped that way

ACT 2

The harness

How we build, verify and operate it, with a coding agent on the team

ACT 3

Production

How every merge reaches production

ACT 1 OF 3

The agent

What runs in production, how one turn works, and the design decisions that survived contact with real traces.

ACT 1

The agent

What runs in production, and why it is shaped that way

ACT 2

The harness

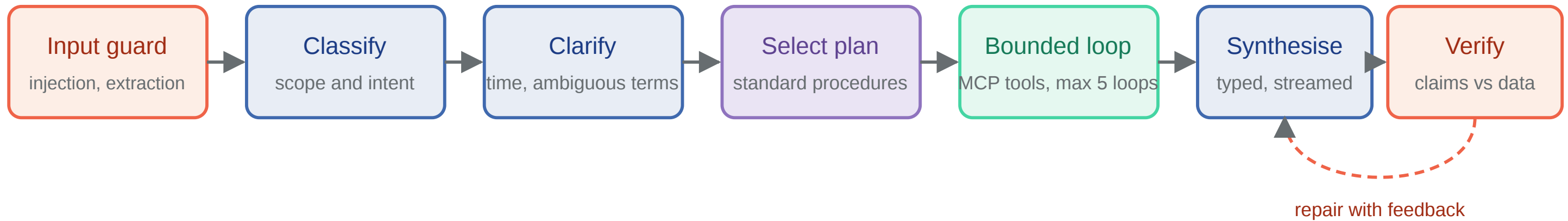
How we build, verify and operate it, with a coding agent on the team

ACT 3

Production

How every merge reaches production

Anatomy of one turn

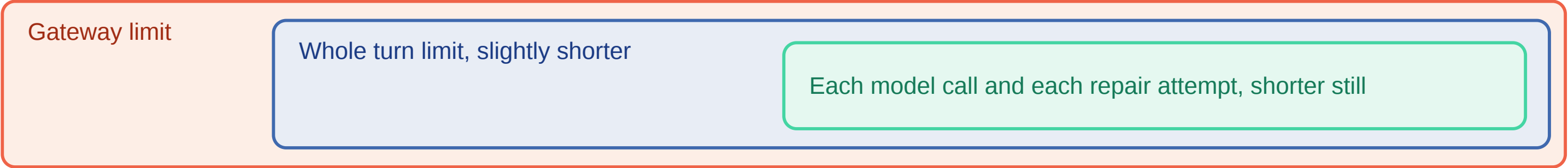


Typed Pydantic state in, typed state out. An error boundary on every box, one span per box, one trace per turn.

Caps: 60 s a model call, 120 s synthesis, 420 s the turn, 430 s the gateway. Inner caps fire first.

- **The model never touches the database.** Tools are discovered and called over MCP: analytics, plus a sandboxed code executor with a pre-flight check on generated Python.
- **Plans, not improvisation.** Known questions route to a curated procedure. Open-ended agency is the fallback, not the default.
- **Stateless per request.** A fresh agent each turn with the user's token baked in. The database is the memory; nothing survives in process.
- **Verification is a stage, not a hope.** A claim extractor matches every number and entity in the narrative back to the data the user can see.

Bounded, verified, fail-closed



Every loop is bounded

A hard cap on iterations, feedback carried into the next attempt, a defined outcome when the budget runs out. No infinite loops, no silent empty answers.

Inner limits fire first

Limits nest from the outside in, so the agent always times out before the gateway does. A timeout arrives as a diagnostic error in the trace, not a dropped connection.

Numbers are verified, failures fail closed

Every quantity passes a range, sign and period check before it streams. Every stage returns typed state from an error boundary, never a stack trace. If a check cannot run, the answer is withheld.

Each of these rules replaced a line in a prompt that said MUST and was ignored under load. A prompt is a polite request. A cap, a schema and a verifier are contracts.

ACT 2 OF 3

The harness

The agent does not ship itself. The harness is the workspace, the skills and the guardrails that let a small team and a coding agent build, verify and operate it.

ACT 1

The agent

What runs in production, and why it is shaped that way

ACT 2

The harness

How we build, verify and operate it, with a coding agent on the team

ACT 3

Production

How every merge reaches production

One workspace for the whole dependency chain

Workspace: a devcontainer today; native cloud-workstation mode in progress on the same scripts

Coding agent + harness CLI

Skills, slash commands, a repo-explorer agent, the rules file
CLI: init, start, doctor, token, plans, evaluate, worktree, cypress
MCP bridge: same data tools as the product, same user token

Submodules, editable installs, the backend reloads on any of them

Agent library

Python, evals, CLI

Chart renderer

Python + React

App

FastAPI + React MFE

Tenant packs

plans, prompts, goldens

Supporting services in Docker Compose, started and health-checked by the CLI

Postgres

app database

Redis

Model gateway

aliases over Vertex

Tracing stack

web, worker, ClickHouse,
object store

Data MCP

governed tools

Code-exec MCP

sandboxed Python

Chart render

headless

- **Why one workspace.** A one-line change in the agent library is only testable through the app. Each repository is an editable install and the backend watches all three source trees, so a save reloads the running app.
- **Why the services run locally.** The full pipeline runs end to end without a deployed environment, and the token command walks the real login flow, so the data tools see a real user's entitlements.

The skills: versioned procedures, organised around the life of a change

A skill is a markdown procedure: a trigger, ordered steps, guardrails. The agent discovers it; an engineer reads it as a runbook. One source of truth, copied to every repository.

BUILD

- **start-ticket**: branch, analyse the ticket, propose, implement, test, summarise
- **architecture, branching, deployment**: the "why", read before touching code

VERIFY

- **evaluate**: the eval suite behind pre-flight checks, resumable per case
- **review-eval**: pass rates, failures handed over with their trace ids
- **pr-review**: a checklist built from our own PR history, plus OWASP Top 10

SHIP

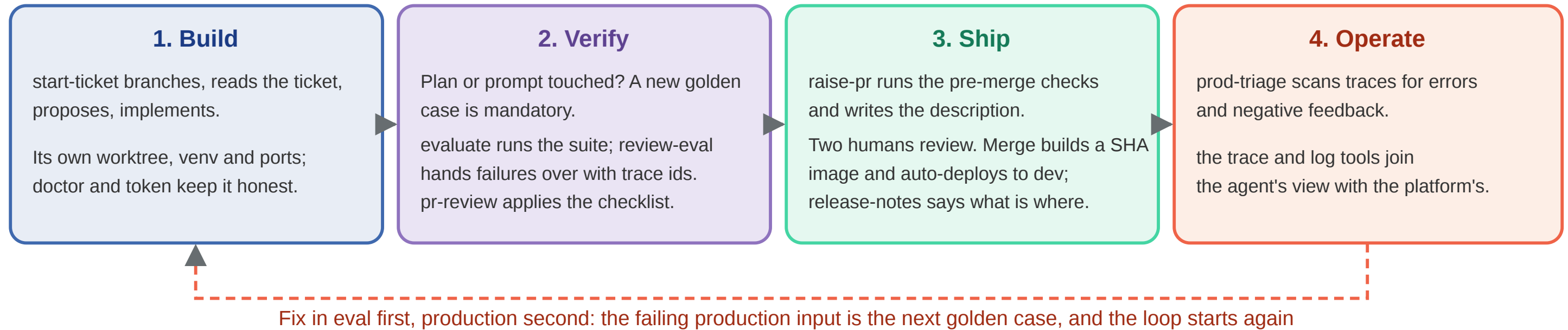
- **raise-pr**: pre-merge checks, push, a description from the ticket and the diff
- **release-notes**: technical and non-technical audiences, across repositories
- **dependency-triage**: red, amber or green by semver and changelog, then gate, smoke test, one y/n, merge

OPERATE

- **prod-triage**: scan production traces for errors and negative feedback, diagnose in parallel, fix when unambiguous
- **trace tools**: pull a trace, render it, walk it span by span
- **query-logs**: platform logs joined to the trace on shared correlation ids

Every skill started as a Slack thread someone had to re-explain. Writing it down once, where the agent reads it, ended the re-explaining.

How we use it: the life of one change



Guardrails on the coding agent

The rules it reads first

- Side effects (merge, push, branch, ticket edits) need explicit instruction in the same turn
- Test-plan checkboxes are human claims. The agent never ticks one.
- Tests come with the code; a prompt, plan or agent-path change comes with a golden case
- Logs are not fixes. "Add logging" means ask for the real one.
- When a bug reveals a missing harness layer, name the layer, not just the patch

Gates it cannot bypass

- Two human reviewers on AI-tagged PRs, five minutes after CI, naming a specific concern
- New-code coverage at or above 80% is blocking; every error path has a test
- Structural limits by lint: 500 lines a file, 50 a function, 5 parameters, complexity 10
- Commit-message and PR-body checks in CI; never merge a Dependabot bump on semver alone

An image it cannot change

- Our own pinned image: the agent CLI version-pinned, auto-update disabled
- Root-owned managed settings the user cannot override; sudo minus shells, su and the agent binary
- Telemetry to the workstation collector, not a sidecar the user can switch off
- Every build publishes an SBOM, then fails on fixable HIGH or CRITICAL

Same sentence, second meaning. The coding agent will fail too. The rules, the review gates and the image are the harness that catches it.

ACT 3 OF 3

Production

Getting an agent to production once is a demo. Getting every merge to every tenant, every fortnight, without drama, is engineering.

ACT 1

The agent

What runs in production, and why it is shaped that way

ACT 2

The harness

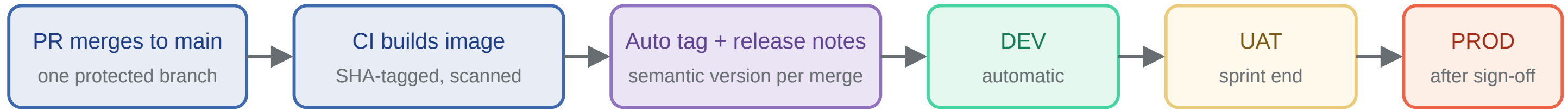
How we build, verify and operate it, with a coding agent on the team

ACT 3

Production

How every merge reaches production

From merge to nine production-grade stacks



Identical code in every environment. Only configuration differs.

Per-tenant packs carry identity, warehouse, model project, tracing and monitors.

The deploy trigger is a one-line image tag change in a separate ops repo.

ArgoCD auto-syncs and self-heals. The app repo builds images; the ops repo decides where they run.

- **Fix forward, never cherry-pick.** A hotfix is a normal merge to main, promoted faster. One code line to reason about.
- **Dev ahead of UAT is by design.** The gap between the dev and UAT tags is the cadence, not drift. A bot reminds us at sprint end.
- **CI gates on the way in:** secrets, dependency review, image scan, static analysis with a blocking coverage gate, commit lint, scheduled load test.
- **Containers run as a numeric non-root user**, enforced by cluster policy. Recorded as an ADR with the ops change it pairs with.



Questions

Blaise Paradeza, Senior AI Software Engineer

Jiggy Kakkad, Principal AI Engineer and Software Engineering Lead

Quantium, Sydney

quantium.com

We are hiring engineers who like harnesses